GYANMANJARI INNOVATIVE UNIVERSITY

Gyanmanjari Institute of Technology – Faculty of Engineering &Technology

Distributed Software Systems

Distributed Chat System using REST APIs, RabbitMQ, Redis and Docker

Student Name	Kothari Tarj Mehulbhai	Enrollment No.	260205020001
Subject	AdvanceSoftwareEngineering (METCE11501)	Semester	1 (M.Tech)
Activity	ALA 2	Marks	10

1. Aim

To design and simulate a distributed application (a chat server) using **REST APIs**, **RabbitMQ** and **microservices with Docker**, and to demonstrate how components communicate across the network and manage distributed tasks. The report contains the architecture diagram, communication flow, code samples and screenshots.

2. Introduction

A distributed system is a set of independent processes running on different nodes that cooperate over a network and appear to the user as one system. Key concerns are **communication**, **scalability**, **fault tolerance** and **loose coupling**. A common approach is to split the application into small services and connect them with synchronous calls (REST/HTTP) and asynchronous messaging (message broker).

Technology	Role in this project
Flask (REST API)	chat-api service: accepts HTTP requests from clients (POST/GET /messages)
RabbitMQ	Messagebroker.DecouplestheAPIfrommessageprocessing;durablequeue 'chat_messages'
Worker (pika consumer)	chat-worker service: consumes queued messages and stores them
Redis	Shared data store holding the chat history list
Docker / Docker Compose	Packages each service in a container and runs all of them on one virtual network

3. System Architecture

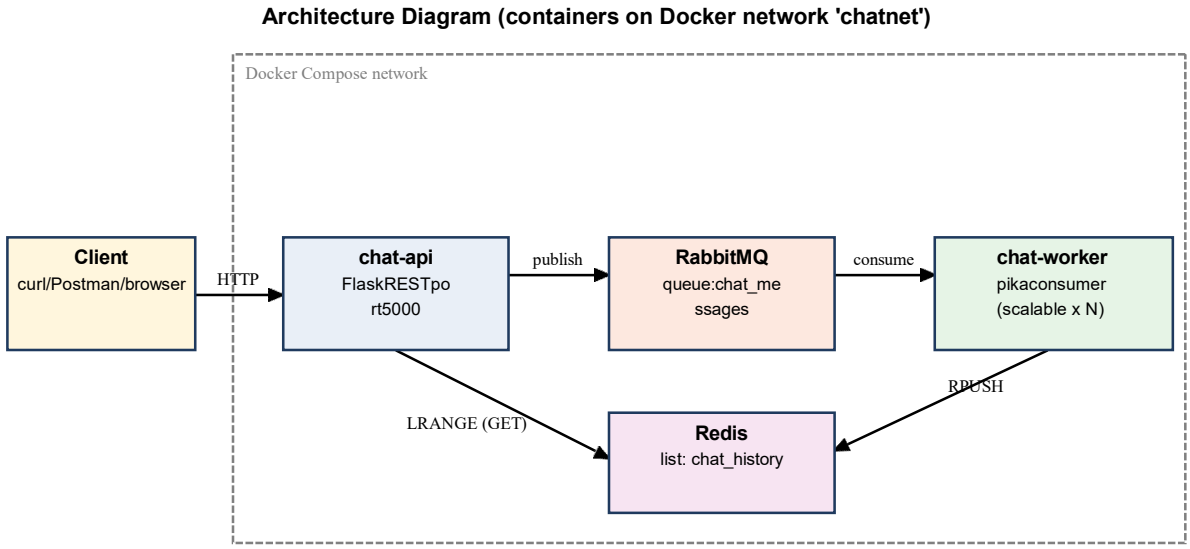


Figure 1: Architecture of the distributed chat system.

Communication flow

Step	From →To	Protocol	Description
1	Client →chat-api	HTTP POST /messages	Client sends {user, text} as JSON.
2	chat-api →RabbitMQ	AMQP	APIvalidatesdata,publishesapersistentmessageand returns **202 Accepted** immediately.
3	RabbitMQ →chat-worker	AMQP	Aworkerreceivesthemessage(round-robinifseveral workers run).
4	chat-worker →Redis	Redis protocol	Workerappendsmessagetolist*chat_history*,then sends an ACK to RabbitMQ.
5	Client →chat-api →Redis	HTTP GET /messages	APIreadsthelast50messagesfromRedisandreturns JSON.

Why asynchronous messaging? The API does not wait for storage; if the worker is slow or temporarily down, messages wait in the durable queue and are not lost. More workers can be added to share the load (distributed task management).

4. Project Structure

```
chat-system/
|--docker-compose.yml
|--api/
|    |--app.py
|    |--requirements.txt
|    `--Dockerfile
`--worker/
     |--worker.py
     |--requirements.txt
     `--Dockerfile
```

5. Implementation (Code Samples)

chat-api: api/app.py

```
importjson,os,time
import pika, redis
fromflaskimportFlask,request,jsonify
```

```
app=Flask(name)
RABBIT_HOST = os.getenv("RABBIT_HOST", "rabbitmq")
REDIS_HOST = os.getenv("REDIS_HOST", "redis")QUEUE
= "chat_messages"

r=redis.Redis(host=REDIS_HOST,port=6379,decode_responses=True)


defpublish(msg:dict):
    """SendonemessagetotheRabbitMQqueue(persistent)."""
    conn = pika.BlockingConnection(pika.ConnectionParameters(host=RABBIT_HOST))
    ch = conn.channel()
    ch.queue_declare(queue=QUEUE, durable=True)
    ch.basic_publish(exchange="", routing_key=QUEUE, body=json.dumps(msg),
                     properties=pika.BasicProperties(delivery_mode=2))
    conn.close()


@app.post("/messages")
def post_message():
    data = request.get_json(force=True, silent=True) or {}
    if not data.get("user") or not data.get("text"):
        returnjsonify(error="'user'and'text'arerequired"),400
    msg = {"user": data["user"], "text": data["text"], "ts": time.time()}
    try:
        publish(msg)
    exceptpika.exceptions.AMQPError:
        return jsonify(error="message broker unavailable"), 503
    return jsonify(status="queued"), 202


@app.get("/messages")
def get_messages():
    items = [json.loads(x) for x in r.lrange("chat_history", -50, -1)]
    return jsonify(items)


@app.get("/health")
def health():
    returnjsonify(status="ok")


if name== "main": app.run(host="0.0.0.0",
    port=5000)
```

chat-worker: worker/worker.py

```
import json, os, time, socket
import pika, redis

RABBIT_HOST = os.getenv("RABBIT_HOST", "rabbitmq")
REDIS_HOST = os.getenv("REDIS_HOST", "redis")QUEUE
= "chat_messages"
WORKER_ID=socket.gethostname()

r=redis.Redis(host=REDIS_HOST,port=6379,decode_responses=True)


def on_message(ch, method, properties, body):
    msg = json.loads(body)
    msg["processed_by"] = WORKER_ID          # shows which worker handled it
    r.rpush("chat_history", json.dumps(msg))
    print(f"[{WORKER_ID}] stored message from {msg['user']}", flush=True)
    ch.basic_ack(delivery_tag=method.delivery_tag)     #ackonlyaftersaving


defmain():
    while True:                                # retry until RabbitMQ is ready
        try:
            conn = pika.BlockingConnection(pika.ConnectionParameters(host=RABBIT_HOST))
            break
        exceptpika.exceptions.AMQPConnectionError:
            print("RabbitMQ not ready, retrying in 3s...", flush=True)
            time.sleep(3)
    ch = conn.channel()
    ch.queue_declare(queue=QUEUE, durable=True)
```

```
    ch.basic_qos(prefetch_count=1)              # fair dispatch between workers
    ch.basic_consume(queue=QUEUE, on_message_callback=on_message)
    print(f"[{WORKER_ID}] waiting for messages...", flush=True)
    ch.start_consuming()


if name== "main": main()
```

Dockerfiles and requirements

```
#api/requirements.txt
flask==3.0.3
pika==1.3.2
redis==5.0.4

#api/Dockerfile
FROMpython:3.11-slim
WORKDIR/app
COPYrequirements.txt.
RUNpipinstall--no-cache-dir
    -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

```
#worker/requirements.txt
pika==1.3.2
redis==5.0.4

#worker/Dockerfile
FROMpython:3.11-slim
WORKDIR/app
COPYrequirements.txt.
RUNpipinstall--no-cache-dir
    -rrequirements.txt
COPY worker.py .
CMD["python","worker.py"]
```

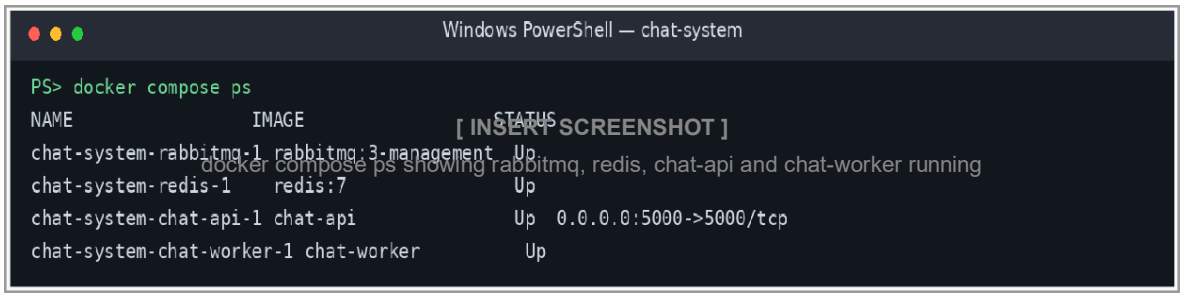
docker-compose.yml

```
services:
  rabbitmq:
    image: rabbitmq:3-management
    ports:
      -"5672:5672"
      - "15672:15672"         # management UI (guest /
  guest)redis:
    image:redis:7
  chat-api:
    build:./api
    ports:
      -"5000:5000"
    environment:
      RABBIT_HOST:rabbitmq
      REDIS_HOST: redis
    depends_on: [rabbitmq, redis]
  chat-worker:
    build:./worker
    environment:
      RABBIT_HOST:rabbitmq
      REDIS_HOST: redis
    depends_on:[rabbitmq,redis]
```

6. Execution and Demonstration

Step 1: Build and start all services

```
docker compose up --build -d
docker compose ps
```



Step 2: Send chat messages (REST)

```
curl-XPOSThttp://localhost:5000/messages-H"Content-Type:application/json"\
     -d'{"user":"Alice","text":"HellofromAlice"}'
curl-XPOSThttp://localhost:5000/messages-H"Content-Type:application/json"\
     -d'{"user":"Bob","text":"HiAlice,thisisBob"}'
```

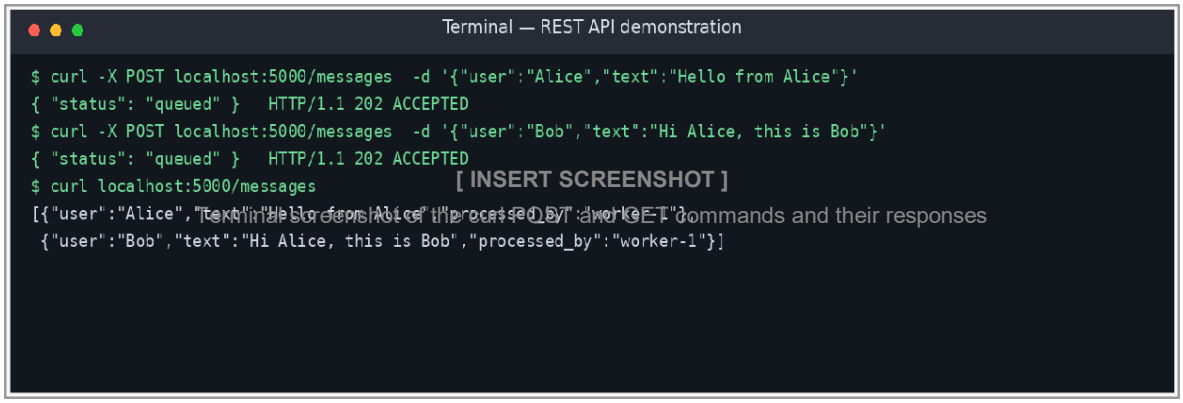
Expected response for each request: `{"status":"queued"}` with HTTP status 202.

Step 3: Read the chat history

```
curlhttp://localhost:5000/messages
```

Expected response (timestamps and worker id will differ):

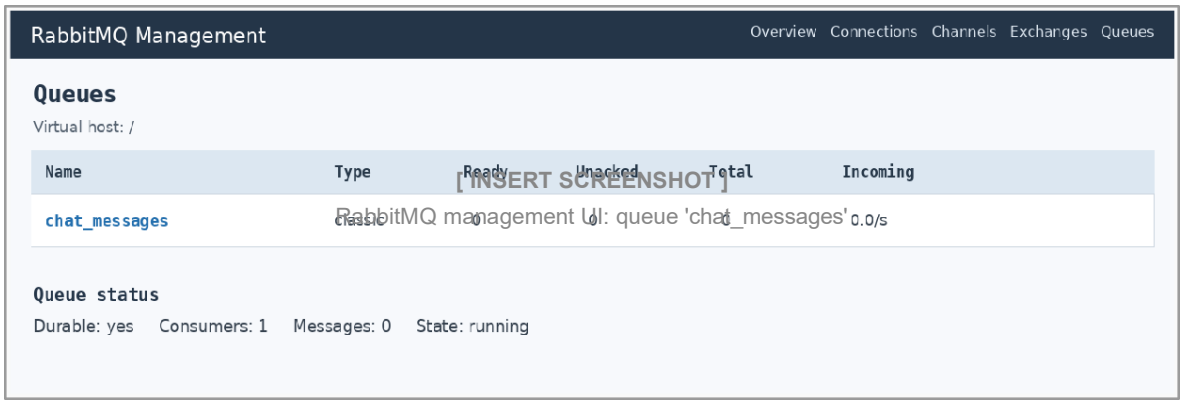
```
[
  {"user":"Alice","text":"HellofromAlice","ts":1.7e9,"processed_by":"3f2a1c9d8e7b"},
  {"user":"Bob","text":"HiAlice,thisisBob","ts":1.7e9,"processed_by":"3f2a1c9d8e7b"}
]
```



Terminal screenshot of the curl commands and their responses

Step 4: Observe the queue in RabbitMQ

Open `http://localhost:15672` (user *guest*, password *guest*) and view the *Queues* tab to see *chat_messages* with message rates.

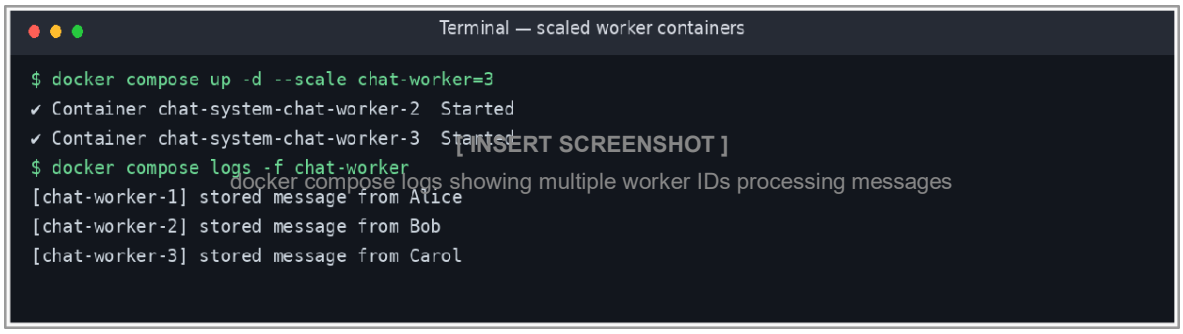


RabbitMQ management UI: queue 'chat_messages'

Step 5: Scale the workers (distributed task handling)

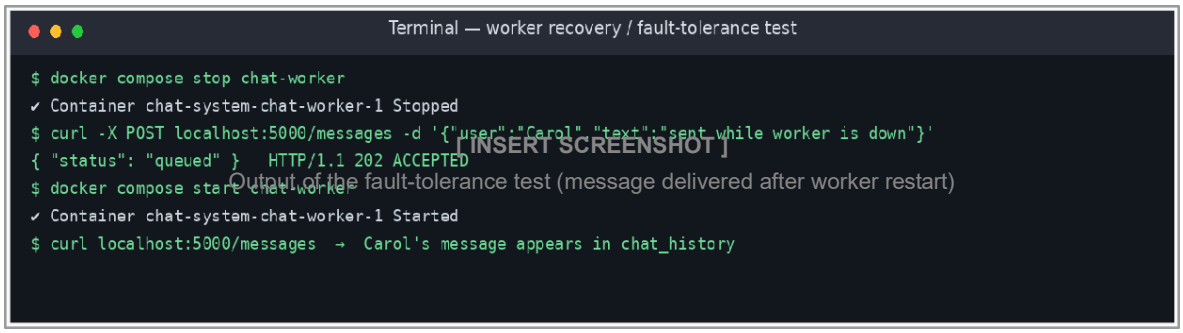
```
docker compose up -d --scale chat-worker=3
docker compose logs -f chat-worker
```

Send several messages again. The logs show different worker container IDs handling different messages, proving that RabbitMQ distributes the tasks across workers.



Step 6: Fault-tolerance test

```
dockercomposestopchat-worker              #workergoesdown
curl-XPOSThttp://localhost:5000/messages-H"Content-Type:application/json"\
     -d '{"user":"Carol","text":"sent while worker is down"}'     # still returns 202
docker compose start chat-worker          # worker returns, queued message is processed
curl http://localhost:5000/messages       # Carol's message now appears
```



7. Result Analysis

Distributed-system property	How it is achieved here
Network communication	HTTP/RESTbetweenclientandAPI;AMQPbetweenservicesandbroker;Redis protocol for storage.
Loose coupling	API and worker never call each other directly; only the queue connects them.
Scalability	Workersscalehorizontallywith--scale;APIcanalsobereplicatedbehindaload balancer.
Fault tolerance	Durablequeue+persistentmessages+manualACK:messagessurviveworker downtime.
Portability	Every service is a Docker image, so it runs identically on any machine.
Limitations	SingleRabbitMQandRedisnode(singlepointsoffailure);noauthenticationonthe API; message order is not guaranteed with multiple workers.

8. Conclusion

A distributed chat application was designed as four cooperating containers. REST gives clients a simple interface, RabbitMQ provides reliable asynchronous communication and task distribution, Redis keeps shared state, and Docker Compose makes the whole system reproducible. The experiments showed message flow across services, load sharing among workers and recovery after a worker failure.

References

- IanSommerville,*SoftwareEngineering*,9thed.,PearsonEducation,2010(Distributedsoftwareengineering).

- RabbitMQ Tutorials (Work Queues), rabbitmq.com; Docker Compose documentation, docs.docker.com.
- Flask documentation, flask.palletsprojects.com; redis-py and pika library documentation.